

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 50%

QUESTIONS: 5

Duration: 1 Hour
Total Marks:50

Annexure A

Scenario Based Assignment

Code of Conduct:

I K.ASWINI(192312397)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

- i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty
- ii. Explain how A* would handle the same situation, including how it balances cost and heuristic
- iii. Analyze the impact of heuristic accuracy on both algorithms
- iv. Discuss how dynamic changes (blocked paths) affect search performance
- v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

2. Scenario: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

- i. Explain how Hill Climbing would work in this scenario and identify its limitations
- ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation
- iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations
- iv. Recommend the best approach and justify your choice based on scalability and adaptability

3. Scenario: An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

- i. Explain why this problem requires an online search agent instead of offline search
- ii. Analyze the challenges of partial observability and dynamic environments

- iii. Describe how the rover builds and updates its internal model
- iv. Compare online search with uninformed and informed search strategies
- v. Justify the most suitable approach considering uncertainty, adaptability, and safety

4. **Scenario:** A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

- i. Clearly define variables, domains, and constraints with explanation
- ii. Explain how backtracking search works in this scenario
- iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency
- iv. Analyze real-world challenges and justify the best strategy for scalability

5. **Scenario:** Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must compete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

- i. Explain how the Minimax algorithm models this problem
- ii. Analyze the computational challenges of large game trees
- iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning
- iv. Design an evaluation function and justify the factors considered
- v. Recommend a strategy for balancing optimality and real-time performance

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrate s clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

ANSWERS

1. AI Drones in Flood-Affected Region

i. Greedy Best-First Search (GBFS)

Greedy Best-First Search selects the path based only on the heuristic $h(n)$, i.e., straight-line distance to the goal.

Behavior:

- Drone always chooses the path that *appears closest* to the destination.
- Ignores actual travel cost (terrain, weather).

Strengths:

- Very fast decision-making (important in emergencies)
- Low computational overhead
- Works well if heuristic is close to reality

Weaknesses:

- Can choose risky or blocked paths
- Not optimal (may not give safest or shortest path)
- Poor in dynamic environments (no cost awareness)

ii. A* Search

A* uses:

$$f(n)=g(n)+h(n)$$

Where:

- $g(n)$ = actual cost (terrain, weather, risk)
- $h(n)$ = heuristic (distance)

Behavior:

- Considers both **current path cost** and **future estimate**
- Avoids dangerous/expensive paths

Advantages:

- Finds optimal path (if heuristic is admissible)
- Balances speed and safety
- Adapts better to real-world costs

iii. Impact of Heuristic Accuracy

Heuristic Quality	Effect on GBFS	Effect on A*
Accurate	Very efficient	Fast & optimal
Overestimate	Misleading	May lose optimality
Underestimate	Still works	Guarantees optimality
Poor heuristic	Very bad decisions	Slower but still reliable

iv. Effect of Dynamic Changes

- **Blocked paths:**
 - GBFS → May repeatedly choose blocked routes
 - A* → Recomputes path considering updated costs
- **Changing costs:**
 - Requires re-planning
 - A* handles this better with updated $g(n)$

v. Recommended Algorithm

A* is best

Justification:

- Ensures **safety (cost awareness)**

- Provides **optimal path**
 - Handles **dynamic changes better**
 - Slightly slower but **more reliable for critical missions**
-

2. Smart Traffic Optimization

- **Objective Function:** Minimize
 - Waiting time
 - Fuel consumption
 - Traffic congestion
- **State:** Signal timings at all intersections
- **Search Space:** Extremely large (combinatorial)

Traditional search is inefficient due to:

- Exponential growth
 - Real-time requirement
 - Continuous updates
-

i. Hill Climbing

Working:

- Start with a solution (signal timings)
- Make small changes
- Move to better solution if improvement exists

Limitations:

- Gets stuck in local optimum
 - Cannot explore globally
 - No backtracking
-

ii. Local Maxima, Plateaus, Ridges

- **Local maxima:**
Solution better than neighbors but not global best
→ Traffic seems optimized but still congested
 - **Plateaus:**
No improvement area
→ Many similar signal timings
 - **Ridges:**
Requires complex moves
→ Small changes don't improve traffic flow
-

iii. Simulated Annealing & Genetic Algorithms

Simulated Annealing

- Sometimes accepts worse solutions
- Escapes local maxima
- Good for dynamic problems

Genetic Algorithm

- Population-based search
- Uses selection, crossover, mutation
- Explores large space efficiently

iv. Recommended Approach

Genetic Algorithm (Best Choice)

Reason:

- Highly scalable
- Works with large search spaces
- Adapts to changing traffic
- Finds near-optimal global solutions

3. Mars Rover (Online Search)

i. Why Online Search?

- Environment unknown
- No complete map
- Decisions must be made in real-time

Offline search is not possible

ii. Challenges

- **Partial observability:** Cannot see entire terrain
- **Dynamic environment:** New obstacles appear
- **Uncertainty:** Sensor limitations

iii. Internal Model

Rover builds:

- Map of explored area
- Updates obstacle positions
- Learns terrain costs

Uses **incremental learning**

iv. Comparison

Type	Limitation
Uninformed Search	No direction → inefficient
Informed Search	Needs full map
Online Search	Works with partial knowledge

v. Best Approach

Online Search (e.g., LRTA*)

Justification:

- Adapts in real-time
- Learns environment
- Safe navigation
- Handles uncertainty

4. Exam Timetable (CSP)

Problem Formulation

Variables:

- Exam slots for each subject

Domains:

- Available time slots & halls

Constraints:

- No overlapping exams
- Limited halls
- Subject ordering
- Faculty availability
- Special accommodations

ii. Backtracking Search

- Assign slot to subject
- Check constraints
- If conflict → backtrack
- Continue until solution found

iii. Constraint Propagation

Forward Checking:

- Removes invalid options early
- Reduces search space

Benefits:

- Faster solution
- Avoids unnecessary computation

iv. Real-world Challenges

- Large number of constraints
- Dynamic changes (faculty/student)
- High complexity

Best Strategy:

- Backtracking + Forward Checking + Heuristics (MRV)

5. Game AI (Minimax & Alpha-Beta)

i. Minimax Algorithm

- Models game as decision tree
- MAX player → maximize score
- MIN player → minimize score

ii. Computational Challenges

- Huge game tree
- Exponential complexity
- Time constraints

iii. Alpha-Beta Pruning

- Eliminates unnecessary branches
- Does not affect final decision

Working:

- Alpha = best for MAX
- Beta = best for MIN
-

Prune when:

- $\alpha \geq \beta$

Reduces computation significantly

iv. Evaluation Function

Should consider:

- Game score
- Position advantage
- Resources
- Risk level

Example:

Evaluation = $w_1(\text{score}) + w_2(\text{position}) + w_3(\text{resources})$

v. Recommended Strategy

Minimax + Alpha-Beta + Heuristic Evaluation

Justification:

- Faster decision making
- Near-optimal results
- Works under time constraints